I initially built a lightweight computer vision pipeline using handcrafted features such as FFT frequency analysis, sharpness, color statistics, and edge distributions combined with Logistic Regression. While this approach was extremely fast (~10 ms/image), it achieved only **81% accuracy** and struggled to generalize across different lighting conditions and camera angles.

I then switched to a pretrained **MobileNetV2** CNN, which improved accuracy to **93%**. During training I observed shortcut learning the model was classifying image content (e.g., grass or sky) instead of screen artifacts. To eliminate this bias, I captured real photos, displayed those exact images on a laptop screen, and photographed them again. This forced the model to learn physical differences between real scenes and screen recaptures rather than scene content.

Finally, I fine-tuned the backbone and combined the CNN output with handcrafted frequency-domain features (FFT, DCT, and noise statistics) using a Logistic Regression late-fusion model. The final hybrid model achieved **96.8% accuracy** on the held-out validation set.

Performance

- **Validation Accuracy: 96.8%**
- **Inference Latency:** ~74 ms/image on an Intel laptop CPU
- **Throughput:** ~13.5 images/second per CPU core

Deployment Cost

For cloud deployment, AWS Lambda is the most cost-effective option. With model caching enabled, inference costs approximately **$2.20 per million images**. Without caching, model loading adds roughly 300 ms per request, increasing both latency and cost by nearly 5×. Since the model is only **8.7 MB**, keeping it loaded in memory is the preferred deployment strategy.

Limitations & Future Work

The dataset was collected using a single phone camera and one laptop display, so the model may still learn hardware-specific characteristics. With more time, I would expand the dataset to include multiple smartphones, OLED/LCD displays, varying lighting conditions, and additional screen types to improve generalization. I would also optimize the model further for mobile deployment using ONNX or TensorRT.